

Image deformation model in CASToR

April 2, 2019

Foreword

CASToR is designed to be flexible, but also as generic as possible. Any new implementation should be thought to be usable in as many contexts as possible; among all modalities, all types of data, all types of algorithms, etc.

Before adding some code to CASToR, it is highly recommended to read the general documentation *CASToR_general_documentation.pdf* to get a good picture of the project, as well as the programming guidelines *CASToR_programming_guidelines.pdf*. Also, the philosophy about adding new modules in CASToR (*e.g.* projectors, optimizers, deformations, dynamic models, etc) is fully explained in *CASToR_add_new_modules.pdf*. Finally, the doxygen documentation is a very good resource to help understanding the code architecture.

1 Summary

This HowTo guide describes how to add a new image transformation class for applications such as motion correction. CASToR can implement image transformations for several types of image-based motion correction, such as physiological motion (respiratory and cardiac), as well as involuntary patient motion. It is assumed that only one type of motion correction can be enabled in a reconstruction, therefore involuntary motion correction cannot be performed with either respiratory or cardiac motion. Simultaneous respiratory and cardiac motion can be performed using a unique set of transformations handling both types of motion.

CASToR is mainly designed to be modular in the sense that adding a new feature should be easy. This guide begins with a rough description of the image deformation part of the CASToR architecture that explains the chosen implementation. Then follows one step-by-step guide that explains how to add a new image deformation by simply adding a new class with few mandatory requirements.

2 Before using any image-based deformation model

2.1 Pre-requisites

Motion management in CASToR differs from the type of motion (physiological or involuntary patient motion). The former requires a prior organization of the raw dataset in *gates* in addition to the deformation parameters, while the latter only requires temporal information and the related transformation vectors.

2.1.1 Physiological motion management (-rm, -cm, -rcm)

Physiological motion includes respiratory and cardiac motions. Due to the cyclic nature of these motions, the CASToR implementation requires a specific organization of the datafile. The following points details the requirements on the dataset and on the characterization of the motion parameters.

Gating: The implementation of image-based correction for these types of motion assumes the elements of the raw datafile (list-mode lines of response, or histogram bins) has previously been sorted into different subsets (*gates*), corresponding to a specific phase of the motion surrogate (e.g, to a respiratory or cardiac phase/amplitude). Several information regarding the gating of the data have to be provided in the reconstruction command-line options using the **-g** option along with a text file containing gating information, in order to enable gating management. The required information for physiological motion correction are the number of respiratory/cardiac gates, and the number of data elements (events) inside each gates. These informations could be given either in the datafile header, or from a separate text file whose path after the -g command-line option (see section 6 for more details).

Figure 1 presents an example of a gating set up file for a dynamic dataset containing 10 frames and 8 (respiratory) gates. The gating associated to each frame must be entered on a separate line. The number of events in each gate are separated by commas. (TODO : Once dual-gating is validated, rather show a file with double gating in order to illustrate with the most complex case). Using the **-g** command-line option and this set up, an image will be reconstructed for each gate within each frame.

```
# -- RESPIRATORY GATING --

nb_respiratory_gates      : 8

respiratory_gates_data_splitting :
1516563,1500341,1743174,1779358,1686703,1642721,1646636,1626992
1466996,1456050,1686886,1722672,1630495,1590838,1591864,1577053
1428700,1417418,1643586,1675922,1590997,1549552,1550657,1536055
1397140,1383691,1604895,1639988,1552764,1512370,1511894,1499491
1374240,1363131,1576879,1605845,1526048,1486712,1488037,1472384
1340110,1326465,1539962,1567528,1490828,1450041,1453035,1434547
1306997,1291690,1496126,1530189,1448835,1411017,1414313,1397439
1280463,1265994,1470635,1501725,1422078,1383648,1386512,1372475
1250712,1232485,1433921,1468570,1390024,1350492,1352661,1340273
2438649,2409871,2794120,2858313,2707544,2631801,2636037,2612423

# -- CARDIAC GATING --

nb_cardiac_gates         : 1

cardiac_gates_data_splitting :
```

Figure 1: Set up of a gating file related to a dynamic dataset with respiratory gating

2.1.2 Involuntary patient motion correction management (-im)

Timestamp based motion: The implementation of image-based patient motion correction requires the timestamp of the transformations to correct the data. During image reconstruction, the image-based transformation operations will be performed when the timestamp of the event is greater than the next timestamp of the deformation. Figure 2 presents an example of a patient motion file indicating the timestamp of each transformation.

```
# -- INVOLUNTARY PATIENT MOTION CORRECTION --  
nb_involuntary_motion_triggers      : 5  
list_involuntary_motion_triggers    : 1200,3450,4220,5560 # (unit = s)
```

Figure 2: Set up of a motion file indicating the timestamp of the transformations for involuntary patient motion. The timestamps must be provided in seconds.

3 Deformation model initialization

A deformation module can be enabled in order to perform image-based transformations between each gate, in order to perform respiratory, cardiac, or both motion corrections. The **-rm**, **-cm**, **-rcm** and **-im** command line options (for respiratory, cardiac, dual-gating, and involuntary patient motion correction respectively) must be used to provide the image deformation information for physiological motion. This option allows to initialize a rigid or elastic deformation model and associated parameters (see details in section 6). CASToR contains a rigid (**deformationRigid**, section3.1) and a non-rigid (**deformationElastic**, section3.2) deformation modules in order to perform image-based transformation, as described below. Details about the available deformation models and information regarding their initialization can be displayed using the command **-help-motion**.

Transformation vectors: During image reconstruction, the image-based motion correction operations will be applied when the code reach a trigger computed from the number elements in each gate (for gated dataset), or from a timestamp (for patient motion). These operations requires a set of *forward* transformations (from a reference position, corresponding to the position in which the images will be reconstructed, to any other position (gate or patient motion time subset) of the dataset), and *backward* transformations (from any position to the reference position), as presented in figure 3. Therefore, for a dataset containing n gates (or patient motion time subsets), $n-1$ forward and $n-1$ backward transformations must be set up.

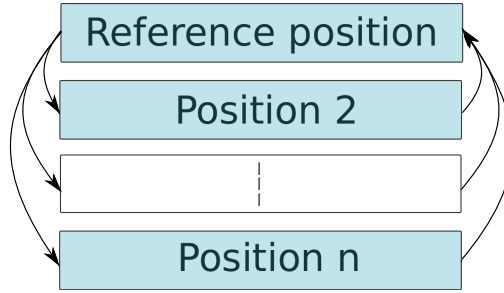


Figure 3: Definition of *forward* (left arrows) and *backward* (right arrows) transformations between the gates of the dataset

3.1 deformationRigid:

This class uses rigid deformations parameterized by 3 translations and 3 rotations in order to perform image-based transformations. As most modules, it can be initialized from the command-line options or using a configuration file (fig 4):

```
-im deformationRigid,tx1,ty1,tz1,ra1,rb1,rc1,tx2,ty2,tz2,ra2,rb2,rc2,...
```

(where tx, ty, tz represents the translations along the different axis, and ra, rb, rc represents the 3 rotations to perform along specific axis (x, y, and z) by default)

```
-im deformationRigid:/path/to/conf/file.txt
```

By default, the transformation parameters correspond to the transformation between the subset (or gate) i to the subset (gate) $i+1$. The forward and backward deformations as defined in figure 3 will be automatically computed from the provided parameters. One can also directly provide the parameters from the reference position to each other position by setting the optionnal parameter *Compute transformations* to 0. Another optional parameter *Rotation convention* defines the axis for the rotation of the 3 angles according to the different conventions (XYZ, ZYX, XYX, etc.. default is XYZ).

3.2 deformationElastic:

This class implements non-rigid deformation using the *Elastix* library[?]. Elastix is a program for image registration which supports many registration methods and various transformation models.

```
Transformation parameters:
1,2,-3.5,-3.5,-2,4.5
3,-0.5,-4.25,3.75,-2,4.25
1,2,1.5,-0.75,-3.5,-3
-0.75,0.5,-2,-4,-3.25,-3
-0.5,3,2.5,1.75,-0.5,-3.5

Rotation convention: XYZ

Compute transformations: 1
```

Figure 4: Example of a configuration file for rigid transformation. Each transformation parameters (3 translations (mm), 3 rotations (degrees)) must be entered on a separate line.

Most part of the Elastix code requires the Insight Segmentation and Registration Toolkit (ITK), so CASToR must be first compiled with both ITK and Elastix in order to be used.

Compilation instructions: TODO Fred (détailler la compilation avec ITK et Elastix)

Initialization: TODO Fred (détailler la configuration et l'utilisation de deformationElastic)

4 The image deformation architecture

The image deformation part of the code is based on 2 main classes: *oDeformationManager* and *vDeformation*. The main program will instantiate and initialize the *oDeformationManager*, and during the iterative reconstruction process, the *oDeformationManager::PerformDeformation()* function will apply the required deformations on the different image matrices when the event counter hits a trigger (a certain number of events in the case of gated dataset, or a certain timestamp for involuntary patient motion). The various steps of application of the deformations on the different image matrices of the algorithm (forward image containing the image to project, backward image containing the correction factors, etc...) is natively managed by the *vDeformation* mother class, so that the developer doesn't need to handle this in his deformation class.

The *vDeformation* class only requires that the child implements the *ApplyDeformations()* function, which will perform the deformation on an input image matrix, and recover the result in an output image matrix, both passed in argument. Please have a look to the section 5 for more help about the deformation model implementation itself.

It is worthy to note that the deformation process itself is NOT multithreaded, therefore a synchronization of the threads will be performed before each image deformation. This is to limit the RAM requirements on some image matrices which should not be multithreaded a-priori.

Regarding the sensitivity image of list-mode reconstruction, the algorithm will generate an image corresponding to the reference position using the *Forward* and *Backward* transformations. The algorithm will simply loop on the total number of transformations, generate a reference image for each set of *Forward* and *Backward* transformation, and average all these images to get a standard reference sensitivity image.

5 Add your own deformation model

5.1 Basic concept

To add a new image-based deformation module, the user only has to build a specific class that inherits from the abstract class *vDeformation*. Then, one just has to implement a set of pure virtual functions for the initialization and application of the model. Please refer to the *CASToR__add_new_modules.pdf* guide in order to fill up the mandatory parts of adding a new module; namely the auto-inclusion mechanism, the interface-related functions and the management functions. Right below are some instructions to help you fill the specific pure virtual projection functions of deformation model.

To ease the implementation, a template class is provided in the source code and already implements all the skeleton. Basically, one will have to change the name of the class and fill the related functions up in his own code. The actual files are *include/image/iDeformationTemplate.hh* and *src/image/iDeformationTemplate.cc* and are actually already part of the source code.

5.2 Implementation of the deformation functions

Several mandatory functions should be implemented (or return an error by default) in a new deformation model class :

ShowHelp(): Simply output some help and guidance to describe what the deformation model does and how to use it.

ReadAndCheckOptionsList(): Implement here the reading of any options specific to this deformation model passed through the argument `const string& a_listOptions`. The user can make use of the *ReadStringOption()* function to parse the list of parameters. If the function is not mandatory (e.g initialization of the model using a file is required rather than with command-line parameters), just send an error message and return 1.

ReadAndCheckConfigurationFile(): Implement here the reading of any configuration file specific to this deformation model, passed through the argument `const string& a_fileOptions`. The user can make use of the *ReadDataASCIIFile()* function to read data from a file. If the function is not mandatory (e.g initialization of the model using command line options is required rather than with a file), just send an error message and return 1.

CheckParameters(): Use this function to check if the private parameters of your class have correctly been initialized. It might be a good idea to initialize all private parameters in the constructor with default erroneous value to properly check their initialization.

Initialize(): Use this function to Instantiate/Initialize any member variables/arrays after the parameters have been checked in the previous function.

ApplyDeformation(): The deformation model must be implemented here, with the help of any private functions if required. This function is described in the pseudocode below. This function has 4 arguments:

`ap_inputImage` : vector containing the input image to deform

`ap_outputImage` : vector in which the transformed image must be recovered

`a_direction` : integer which indicates the direction of the deformation to perform, i.e:

FORWARD_DEFORMATION (from the reference position to the `a_defIdx` position),

BACKWARD_DEFORMATION (from the `a_defIdx` position to the reference position).

The integers FORWARD_DEFORMATION and BACKWARD_DEFORMATION are defined in the beginning of *vDeformation.hh* as macros

`a_defIdx` : defines the index of the transformation

All information and the tools needed to implement these functions are fully described in the template source file *src/projector/iDeformationTemplate.cc*, so please refer to it.

Listing 1: Description of ApplyDeformations function.

```
1
2 int iDeformationTemplate::ApplyDeformations(FLTNB* ap_inputImage,
3                                             FLTNB* ap_outputImage,
4                                             string a_direction,
5                                             int a_defIdx)
6 {
7     // This function is an implementation of the pure virtual mother ↵
8     // function. The actual deformation should be implemented here, with the ↵
9     // help of any private functions if required
10
11     // IMAGE DIMENSIONS:
12     // For code efficiency and readability, the spatial index of a voxel is ↵
13     // a cumulative 1D index.
14     // That is to say, given a voxel [indexX,indexY,indexZ],
15     // its cumulative 1D index is computed by 'index = indexZ*nbVoxXY + ↵
16     // indexY*nbVoxX + indexX'.
17
18     // The image dimensions can be recovered from the ↵
19     mp_ImageDimensionsAndQuantification class
20     // Total number of voxels      :
21     mp_ImageDimensionsAndQuantification->GetNbVoxXYZ()
22     // Number of voxels in a slice  :
23     mp_ImageDimensionsAndQuantification->GetNbVoxXY()
24     // Number of voxels on the X-axis :
25     mp_ImageDimensionsAndQuantification->GetNbVoxX()
26
27     // Any error should return a value >0.
28
29     return 0;
30 }
```

6 Meta-data command line options

List of the different set of options related to image-based deformations :

- **-g *file***: Give a text file defining the dynamic data splitting (due to respiratory/cardiac gating or involuntary patient motion correction). The file should contain the some of the following keywords :
 - nb_respiratory_gates**: number of respiratory gates in the data
 - respiratory_gates_data_splitting**: enter the number of events within each gate, separated by commas. If the data contains several frame (dynamic acquisition), the data splitting of each frame should be entered on a new line
 - nb_cardiac_gates**: number of cardiac gates in the data
 - cardiac_gates_data_splitting**: enter the number of events within each gate, separated by commas. If the data contains several frame (dynamic acquisition), the data splitting of each frame should be entered on a new line.
 - nb_involuntary_motion_triggers**: number of involuntary patient motion triggers in the data
 - listt_involuntary_motion_triggers**: enter the timestamp of each transformation, separated by commas. No specific nomenclature is required for dynamic acquisitions (data containing several frames). However, if the data contains several bed positions, the data splitting of each bed should be entered on a new line.
- **-rm *param***: Give the deformation to be used for respiratory motion correction, along with a configuration file (deformationModel:file), or the list of parameters associated to the deformation model (deformationModel,param1,param2,...).
- **-cm *param***: Give the deformation to be used for cardiac motion correction, along with a configuration file (deformationModel:file), or the list of parameters associated to the deformation model (deformationModel,param1,param2,...).
- **-rcm *param***: Give the deformation to be used for dual gating motion correction (both respiratory and cardiac), along with a configuration file (deformationModel:file), or the list of parameters associated to the deformation model (deformationModel,param1,param2,...).
- **-im *param***: Give the deformation to be used for patient motion correction, along with a configuration file (deformationModel:file), or the list of parameters associated to the deformation model (deformationModel,param1,param2,...).